



Mini-projet GLPI

Gestion du Parc Informatique de CY Tech

SAIDI Bisma
SAIL Rayan
TOUNKARA Mohamed
ALLOCHON Matthieu

CY Tech - ING2

Traitement & Administration de données - 2025-2026

Table des matières

Table des matières.....	1
1. Introduction.....	2
2. Démarche de Reverse Engineering de GLPI.....	3
2.1 Présentation de GLPI.....	3
2.3 Tables GLPI analysées.....	4
2.4 Défauts identifiés dans le modèle GLPI.....	5
3. Nouvelle Modélisation.....	5
3.1 Principes de conception.....	5
3.2 Tables du modèle.....	6
3.3 Schéma relationnel.....	6
3.4 Contraintes d'intégrité.....	8
3.5 Justification des choix techniques.....	8
4. Implementation Oracle.....	9
4.1 Tablespaces.....	9
4.2 Utilisateurs Oracle et rôles.....	9
4.3 Cluster Oracle.....	10
4.4 Index.....	10
5. Développements PL/SQL.....	11
5.1 Triggers.....	11
5.2 Fonctions PL/SQL.....	12
5.3 Procédures avec curseurs.....	12
6. Base de Données Répartie (BDDR).....	13
6.1 Architecture.....	13
6.2 Database Links.....	14
6.3 Vues globales.....	15
6.4 Stratégie de fragmentation.....	15
6.5 Vues matérialisées.....	15
7. Tests de Performance.....	16
7.1 Jeu de données.....	16
7.2 Résultats des requêtes.....	16
7.3 Analyse comparative.....	18
7.4 Comparaison Cergy vs Pau.....	19
7.5 Résultats BDDR - Vision globale.....	20
7.6 Vues Matérialisées vs Vues classiques.....	21
7.7 Limites du projet.....	23
8. Conclusion.....	23

1. Introduction

Dans le cadre du module de Traitement et Administration de Données, ce mini-projet a pour objectif la conception et l'implémentation d'une base de données Oracle inspirée du logiciel open source GLPI, utilisé pour la gestion de parc informatique et le support technique.

GLPI est une solution particulièrement complète reposant sur une base de données comportant plus de 250 tables. Bien que très puissante, cette architecture généraliste présente certaines limites dans un contexte pédagogique et multi-sites comme celui de CY Tech, notamment en matière de lisibilité, de performances et de séparation des données entre les différents campus.

L'objectif de ce projet a donc été de mener un travail de reverse engineering afin d'analyser le fonctionnement de GLPI, puis de proposer une nouvelle modélisation adaptée aux besoins spécifiques de CY Tech. Nous avons conçu une architecture simplifiée mais cohérente, centrée sur deux sites principaux : Cergy et Pau. Cette architecture repose sur une base de données répartie (BDDR) avec une fragmentation horizontale des données par site, complétée par une consolidation globale à l'aide de vues Oracle et de database links.

Le projet couvre plusieurs aspects avancés de l'administration de bases de données Oracle, notamment :

- la création de schémas et de tablespaces ;
- l'optimisation des performances à l'aide d'index et de clusters ;
- l'implémentation de triggers et de procédures PL/SQL ;
- la gestion de l'intégrité métier ;
- la génération de données massives ;
- ainsi que l'analyse des performances via les plans d'exécution Oracle.

À travers ce travail, nous avons cherché à concevoir une solution plus adaptée à un environnement universitaire multi-sites, tout en appliquant de manière concrète les notions d'architecture de bases de données, d'optimisation et d'administration Oracle étudiées en cours.

2. Démarche de Reverse Engineering de GLPI

2.1 Présentation de GLPI

GLPI (Gestion Libre de Parc Informatique) est une solution open source largement utilisée pour la gestion de parc informatique, le suivi des équipements et la gestion des incidents via un système de helpdesk. Développé initialement pour répondre aux besoins des services informatiques, GLPI permet de centraliser les informations liées aux utilisateurs, aux matériels, aux logiciels, aux réseaux ainsi qu'aux tickets d'assistance.

L'application repose sur une base de données MySQL particulièrement riche, composée de plus de 250 tables interconnectées. Ce choix est cohérent pour un logiciel générique utilisé dans des contextes très variés. Cependant, dans le cadre d'un environnement universitaire plus ciblé comme celui de CY Tech, cette architecture entraîne une forte complexité structurelle ainsi que des requêtes parfois coûteuses à exécuter en raison du grand nombre de jointures nécessaires.

Malgré cette complexité, cette organisation offre une grande flexibilité fonctionnelle et permet à GLPI de couvrir un très large périmètre métier.

Dans le cadre de ce projet, GLPI a servi de base d'étude pour comprendre l'organisation d'un système réel de gestion de parc informatique. L'objectif n'était pas de reproduire l'intégralité du modèle existant, mais d'en extraire les éléments essentiels afin de concevoir une version simplifiée, optimisée et adaptée aux besoins spécifiques de CY Tech.

2.2 Méthode d'analyse

Pour analyser GLPI, nous avons commencé par étudier le dépôt officiel du projet ainsi que la documentation destinée aux développeurs. Cette étape nous a permis de mieux comprendre la structure générale de la base de données et son fonctionnement.

Nous avons ensuite identifié les principales conventions utilisées dans GLPI, notamment les noms de tables préfixés par `glpi_`, l'utilisation récurrente de la clé primaire `id`, ainsi que la présence de nombreuses tables d'association servant à gérer les relations entre les différentes entités.

Étant donné le très grand nombre de tables présentes dans la base GLPI, nous avons choisi de concentrer notre étude sur les éléments les plus pertinents pour notre projet : les utilisateurs, les rôles, les équipements matériels, les localisations, le réseau, les VLAN ainsi que les tickets.

Cette analyse nous a permis de mettre en évidence certaines limites du modèle GLPI dans un contexte multi-sites tel que celui de CY Tech. À partir de ces constats, nous

avons conçu un modèle plus simple et mieux adapté à nos besoins, centré sur les sites de Cergy et de Pau, avec la table SITE comme élément principal de structuration.

2.3 Tables GLPI analysées

Parmi les 250 tables GLPI, les tables suivantes ont été identifiées comme pertinentes pour notre périmètre :

Domaine	Tables GLPI	Rôle
Utilisateurs	glpi_users, glpi_profiles, glpi_profiles_users	Utilisateurs, profils et droits
Matériels	glpi_computers, glpi_states, glpi_locations	Inventaire, etats et localisations
Réseau	glpi_networkports, glpi_vlans, glpi_ipaddresses	Ports, VLAN et adresses IP
Tickets	glpi_tickets, glpi_itilcategories	Incidents et demandes
Localisation	glpi_entities, glpi_locations	Séparation organisationnelle

2.4 Défauts identifiés dans le modèle GLPI

L'analyse de GLPI a mis en évidence plusieurs limites pour notre cas d'usage :

Défaut identifié	Conséquence	Solution adoptée
Nombre élevé de tables (250+)	Nombreuses jointures pour des requêtes simples	Modèle cible restreint et indexé
Relations polymorphes itemtype/items_id	Requêtes moins lisibles et optimisables	Clés étrangères explicites
Multi-entités logique, pas physique	Données Cergy/Pau non séparées physiquement	Fragmentation horizontale par site_id + BDDR
Index non alignés sur usages CY Tech	Full table scan sur requêtes fréquentes	Index composés site+type+statut
Pas de traçabilité des affectations	Impossible de retrouver l'historique	Table HISTO_AFFECTATION + trigger

Cette analyse nous a permis de concevoir une architecture plus simple, mieux adaptée au contexte multi-sites de CY Tech et davantage optimisée pour les requêtes les plus fréquentes liées à la gestion du parc informatique.

3. Nouvelle Modélisation

3.1 Principes de conception

Le modèle proposé reprend les fonctionnalités essentielles de GLPI tout en simplifiant son architecture afin de mieux répondre aux besoins du projet. L'objectif principal était de construire une base plus lisible et plus facile à administrer dans un contexte universitaire.

L'architecture repose sur une séparation des données entre les sites de Cergy et de Pau. Chaque campus possède son propre schéma Oracle (glpi_cergy et glpi_pau) contenant les mêmes tables mais avec des données distinctes. Un troisième schéma, GLPI_GLOBAL, permet de centraliser les informations grâce à des database links et des vues UNION ALL.

La colonne site_id a été intégrée dans les principales tables afin d'identifier l'origine des données et de faciliter leur organisation au sein de l'architecture répartie.

3.2 Tables du modèle

Notre modèle comporte 10 tables par site, inspirées du reverse engineering de GLPI :

Table	Table GLPI d'origine	Description
USERS	glpi_users, glpi_profiles_users	Utilisateurs avec rôle et localisation
LOCATIONS	glpi_locations	Bâtiment et salle physique
COMPUTERS	glpi_computers, glpi_states	Parc informatique avec type et statut
SOFTWARES	glpi_softwares	Logiciels et licences
COMPUTER_SOFTWARES	glpi_items_softwares	Association matériel-logiciel
VLAN	glpi_vlans	Réseaux logiques par site
NETWORKS	glpi_networknames	Plages IP rattachées aux VLANs

EQUIPEMENT_RESEAU	glpi_networkequipments	Switchs, routeurs, bornes Wi-Fi
TICKETS	glpi_tickets	Incidents avec priorité et statut
HISTO_AFFECTATION	glpi_logs (inspire)	Historique des affectations materiel-user

3.3 Schéma relationnel

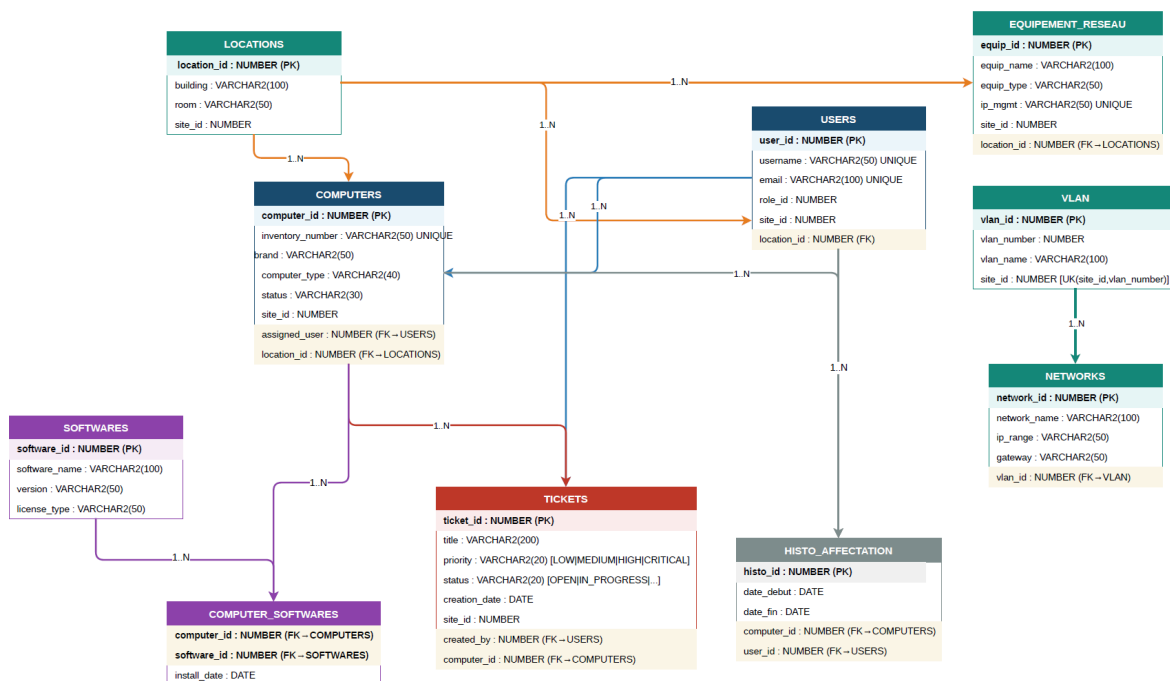


Schéma relationnel — Modèle cible CY Tech inspiré du reverse engineering de GLPI (10 tables par site, clés primaires, clés étrangères et cardinalités 1..N)

Le schéma relationnel présenté ci-dessus traduit le modèle UML sous forme de tables Oracle, en définissant les clés primaires, les clés étrangères ainsi que les différentes contraintes d'intégrité.

La table LOCATIONS occupe un rôle central dans la gestion de la localisation physique des ressources. Elle est référencée par les tables USERS, COMPUTERS et EQUIPEMENT_RESEAU à travers la clé étrangère location_id.

La table USERS représente l'entité principale des acteurs du système. Elle est utilisée par la table COMPUTERS pour identifier l'utilisateur responsable d'un équipement, ainsi que par la table TICKETS pour enregistrer le créateur du ticket et le technicien assigné.

La table COMPUTERS constitue le cœur du parc informatique. Elle est liée à TICKETS, afin d'associer les incidents au matériel concerné, à HISTO_AFFECTATION pour assurer la traçabilité des affectations, ainsi qu'à COMPUTER_SOFTWARES pour gérer les logiciels installés sur chaque poste.

La partie réseau est organisée autour des tables VLAN et NETWORKS, permettant de gérer les réseaux logiques et les plages d'adresses IP associées. La table EQUIPEMENT_RESEAU regroupe les différents équipements de l'infrastructure tels que les switches, routeurs et bornes Wi-Fi.

Les principales tables intègrent également la colonne site_id, utilisée pour distinguer les données propres à chaque campus dans l'architecture répartie.

3.4 Contraintes d'intégrité

Des contraintes CHECK et UNIQUE ont été ajoutées sur certaines colonnes critiques afin de garantir l'intégrité et la cohérence des données au sein de la base.

Les principales contraintes mises en place sont les suivantes :

- **TICKETS.priority :**
CHECK (priority IN ('LOW', 'MEDIUM', 'HIGH', 'CRITICAL'))
Cette contrainte limite les niveaux de priorité des tickets à des valeurs prédéfinies afin d'éviter l'insertion de valeurs invalides.
- **TICKETS.status :**
CHECK (status IN ('OPEN', 'IN_PROGRESS', 'RESOLVED', 'CLOSED'))
Elle garantit que l'état d'un ticket correspond uniquement aux statuts autorisés par l'application.
- **VLAN :**
UNIQUE (site_id, vlan_number)
Cette contrainte assure qu'un même numéro de VLAN ne peut pas être dupliqué sur un même site, tout en autorisant l'utilisation du même numéro sur des sites différents.
- **EQUIPEMENT_RESEAU.ip_mgmt :**
UNIQUE (ip_mgmt)
Cette contrainte garantit qu'aucun équipement réseau ne possède la même adresse IP de gestion qu'un autre équipement, assurant ainsi l'unicité des adresses d'administration au sein de l'infrastructure.

3.5 Justification des choix techniques

Les différents choix techniques réalisés dans le cadre du projet ont été guidés par les besoins spécifiques d'un environnement universitaire multi-sites ainsi que par les problématiques de performance observées lors du reverse engineering de GLPI.

La fragmentation horizontale par site_id a été retenue afin de séparer physiquement les données des campus de Cergy et de Pau, permettant ainsi d'améliorer les performances des accès locaux et de simplifier l'administration des données.

Les index composés ont été conçus en fonction des requêtes les plus fréquentes du contexte CY Tech, notamment les recherches multicritères portant sur le site, le type de matériel ou le statut des tickets.

L'utilisation d'un cluster Oracle entre les tables COMPUTERS et TICKETS permet d'optimiser les jointures fréquemment utilisées dans les opérations de helpdesk en rapprochant physiquement les données liées.

Les database links et les vues globales assurent une consolidation transparente des informations des différents sites tout en conservant une séparation logique et physique des données.

Enfin, les vues matérialisées ont été mises en place afin d'optimiser les requêtes de reporting global en évitant les accès distants répétés via les database links.

4. Implementation Oracle

4.1 Tablespaces

Quatre tablespaces ont été créés pour séparer les données selon leur nature et leur site :

Tablespace	Taille initiale	Contenu
TS_CERGY_DATA	50 Mo (max 500 Mo)	Tables du schéma glpi_cergy
TS_PAU_DATA	50 Mo (max 500 Mo)	Tables du schéma glpi_pau
TS_INDEX	30 Mo (max 200 Mo)	Tous les index des deux sites
TS_HISTO	20 Mo (max 200 Mo)	Tables HISTO_AFFECTATION

Séparer les index dans TS_INDEX permet de les gérer indépendamment des données et d'optimiser les sauvegardes. TS_HISTO isole les données d'historique qui grossissent rapidement.

4.2 Utilisateurs Oracle et rôles

Quatre utilisateurs Oracle ont été créés afin de séparer clairement les responsabilités et les accès au sein de l'architecture :

- **glpi_global** : administrateur global disposant des droits de lecture et d'écriture sur les deux sites ;
- **glpi_cergy** : utilisateur associé au site de Cergy, avec accès uniquement à son propre schéma ;
- **glpi_pau** : utilisateur associé au site de Pau, avec accès uniquement à son propre schéma ;
- **glpi_readonly** : utilisateur dédié à la consultation et au reporting, avec un accès en lecture seule.

Afin de simplifier la gestion des permissions, trois rôles Oracle ont également été définis :

- **role_admin_global** : droits avancés d'administration (*CREATE SESSION*, *CREATE TABLE*, *CREATE VIEW*, *CREATE PROCEDURE*, etc.) ;
- **role_technicien** : accès standard à la base via *CREATE SESSION* ;
- **role_lecture** : accès limité à la consultation des données.

L'utilisateur **glpi_readonly** ne possède pas de tables propres. Il correspond uniquement à un profil Oracle auquel le rôle **role_lecture** a été attribué. Ce compte peut consulter les tables des schémas **glpi_cergy** et **glpi_pau** grâce aux privilèges *SELECT* accordés, ainsi que les vues consolidées du schéma **glpi_global**.

Cette organisation permet d'appliquer le principe du moindre privilège : chaque utilisateur dispose uniquement des droits nécessaires à ses besoins. Elle garantit également une meilleure sécurité des données en limitant les risques de modification accidentelle lors des opérations de reporting ou d'audit.

4.3 Cluster Oracle

Un cluster Oracle est une structure de stockage physique permettant de regrouper dans les mêmes blocs disques les lignes de plusieurs tables partageant une même clé. Cette organisation améliore les performances des requêtes qui accèdent fréquemment à des données liées.

Sans cluster, Oracle stocke les tables **COMPUTERS** et **TICKETS** séparément. Pour retrouver tous les tickets associés à un ordinateur, le moteur doit effectuer plusieurs opérations : accéder à la table **COMPUTERS** pour identifier l'ordinateur concerné, puis parcourir la table **TICKETS** afin de rechercher les lignes correspondant au bon **computer_id**.

Avec le cluster **cergy_cluster_comp_ticket**, les lignes des tables **COMPUTERS** et **TICKETS** partageant le même **computer_id** sont stockées physiquement au même emplacement disque. Lorsque Oracle accède aux informations d'un ordinateur, les tickets associés sont alors immédiatement disponibles dans le même bloc de données, ce qui réduit fortement les accès disques supplémentaires.

Ce choix est particulièrement pertinent dans le contexte d'un helpdesk GLPI, où l'une des requêtes les plus fréquentes consiste à rechercher tous les tickets liés à un poste informatique. Le cluster optimise donc directement la jointure :

“ COMPUTERS JOIN TICKETS ON computer_id ”

qui constitue une opération centrale dans la gestion des incidents et du support utilisateur.

4.4 Index

Afin d'améliorer les performances de la base de données, plusieurs index ont été créés sur les colonnes les plus sollicitées dans les requêtes SQL. L'objectif principal est de réduire le temps d'accès aux données et d'éviter les parcours complets des tables (*full table scan*) lors des opérations de recherche, de filtrage ou de tri.

Dans un premier temps, des index simples ont été définis sur certaines colonnes utilisées fréquemment dans les clauses WHERE, les recherches ou les tris :

- idx_cergy_users_email et idx_cergy_users_username afin d'accélérer l'identification et la recherche des utilisateurs ;
- idx_cergy_computers_inventory et idx_cergy_computers_status pour retrouver rapidement un équipement selon son numéro d'inventaire ou son état ;
- idx_cergy_tickets_status, idx_cergy_tickets_priority et idx_cergy_tickets_creation pour optimiser les traitements liés aux tickets, notamment les recherches par statut, priorité ou date de création.

En complément, des index composés ont été mis en place sur plusieurs colonnes combinées. Ce type d'index est particulièrement pertinent dans un environnement multi-sites, où les requêtes filtrent fréquemment les données selon le campus, le type de matériel ou l'état des éléments :

- COMPUTERS(site_id, computer_type, status) pour rechercher efficacement les matériels d'un certain type sur un site donné ;
- TICKETS(site_id, status) afin d'afficher rapidement les tickets ouverts ou en cours pour chaque campus ;
- USERS(site_id, status) pour lister les utilisateurs actifs d'un site spécifique ;
- VLAN(site_id, vlan_number) afin de garantir l'unicité des VLANs par site tout en accélérant les recherches réseau ;
- HISTO_AFFECTATION(computer_id, date_debut) et HISTO_AFFECTATION(user_id, date_debut) pour optimiser la consultation chronologique des historiques d'affectation.

Les index ont été choisis en fonction des requêtes les plus utilisées dans GLPI et des besoins du projet. Ils permettent d'améliorer significativement les performances des opérations de consultation et de gestion des données, tout en conservant une structure cohérente et évolutive de la base de données.

5. Développements PL/SQL

5.1 Triggers

Un trigger est un bloc PL/SQL qui s'exécute automatiquement en réponse à un événement sur une table : INSERT, UPDATE ou DELETE. Contrairement à une procédure qui doit être appelée explicitement, le trigger se déclenche de manière transparente sans intervention de l'utilisateur.

Nous avons implémenté trois triggers sur chaque site pour trois raisons distinctes

Trigger 1 - trg_cergy_ticket_creation :

Force la date de création à SYSDATE à chaque INSERT sur TICKETS, quelle que soit la valeur fournie. Cela garantit l'intégrité de la date d'enregistrement.

Trigger 2 - trg_cergy_affectation_histo :

Se déclenche après chaque modification du assigned_user d'un ordinateur. Il ferme automatiquement l'affectation précédente dans HISTO_AFFECTATION et crée une nouvelle entrée. Ce trigger garantit la traçabilité complète des affectations matérielles.

Trigger 3 - trg_cergy_ticket_no_delete_open :

Interdit la suppression d'un ticket en statut OPEN et lève une erreur applicative (ORA-20001). Ce trigger protège l'intégrité du métier : un ticket ouvert ne peut pas disparaître sans avoir été traité.

5.2 Fonctions PL/SQL

Deux fonctions ont été créées sur chaque site :

Fonction nb_tickets_ouverts(p_user_id) :

Retourne le nombre de tickets en statut OPEN ou IN_PROGRESS pour un utilisateur donné. Utilisé dans les requêtes de reporting pour identifier les utilisateurs avec le plus de tickets en attente.

Fonction nb_materiel_actif(p_site_id) :

Retourne le nombre de matériels en statut ACTIVE pour un site donné. Utilisée dans le tableau de bord pour afficher l'état du parc par site.

5.3 Procédures avec curseurs

Deux procédures ont été implémentées sur chaque site :

Procédure add_ticket :

Créer un nouveau ticket après avoir vérifié via un curseur explicite que le technicien assigné existe et possède bien le rôle id = 2. Si le technicien n'est pas trouvé, une erreur applicative est levée (ORA-20002).

Procédure rapport_tickets_priorite :

Utilise un curseur explicite OPEN/FETCH/CLOSE pour parcourir les tickets groupés par priorité et afficher un rapport formaté via DBMS_OUTPUT. Le curseur calcule le nombre de tickets dans chaque statut (OPEN, IN_PROGRESS, RESOLVED, CLOSED) pour chaque niveau de priorité.

5.4 Procédure de génération de données

Une procédure generate_all_data_cergy/pau a été implémentée pour générer des données massives et réalistes. Elle crée en une seule exécution :

- 10 localisations (bâtiments et salles)
- 10 VLANs numérotés
- 20 équipements réseau (switchs, routeurs, bornes Wi-Fi, firewalls)
- 5 000 utilisateurs avec rôles réalistes (1 admin tous les 50, 1 tech tous les 10)
- 1 000 ordinateurs avec marques, modèles et types variés
- 50 000 tickets avec priorités et statuts distribués uniformément
- 100 entrées d'historique d'affectation

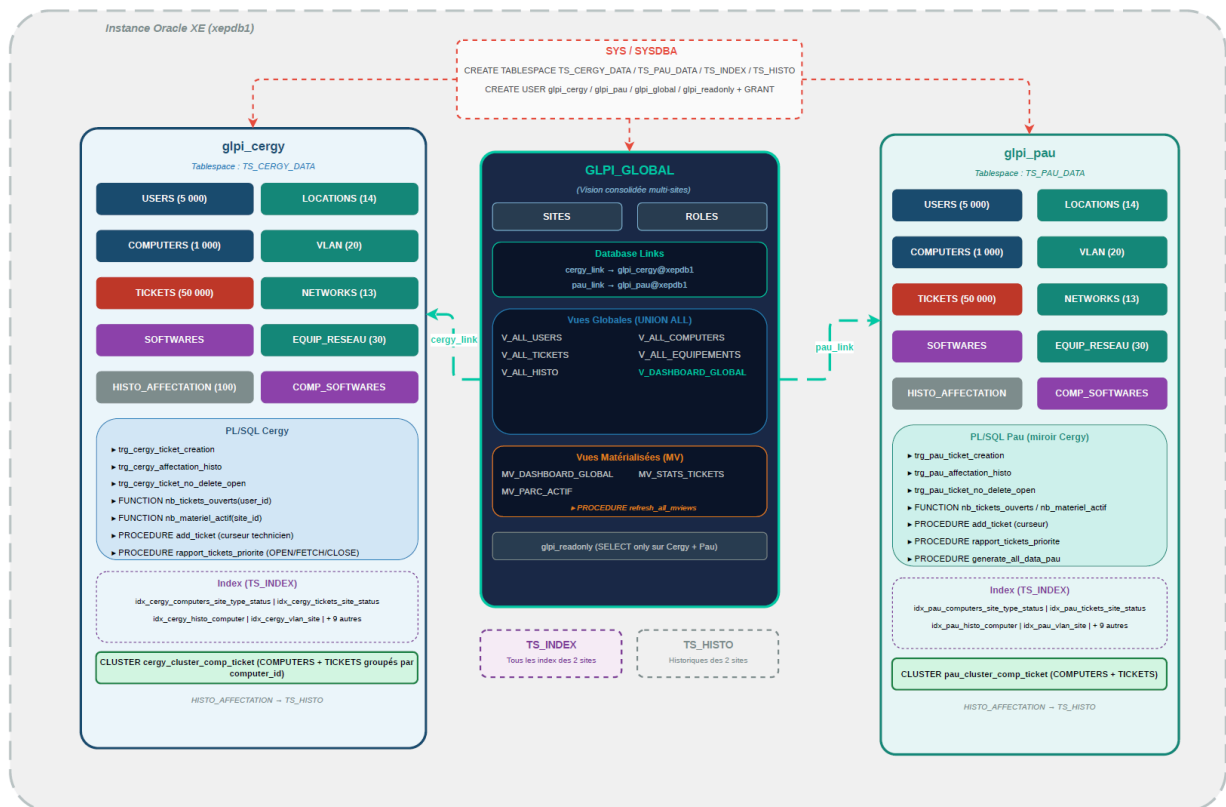
La procédure utilise des commits intermédiaires tous les 100/500 enregistrements pour éviter les locks Oracle et garantir la robustesse en cas d'interruption.

6. Base de Données Répartie (BDDR)

6.1 Architecture

L'architecture BDDR repose sur trois schémas Oracle sur la même instance (xepdb1) :

- glpi_cergy : schéma du site de Cergy, tablespace TS_CERGY_DATA
- glpi_pau : schéma du site de Pau, tablespace TS_PAU_DATA
- GLPI_GLOBAL : schéma global contenant uniquement les vues consolidées et les database links



Architecture BDDR — Les trois schémas Oracle (glpi_cergy, glpi_pau, GLPI_GLOBAL) avec database links, vues globales et vues matérialisées

Le principe est de séparer les données de chaque campus afin de limiter les accès inutiles et de simplifier l'administration des informations locales. Chaque site travaille donc principalement sur son propre schéma, tandis que GLPI_GLOBAL permet d'obtenir une vue consolidée de l'ensemble du parc informatique grâce aux vues UNION ALL.

Dans le cadre de notre projet, cette organisation permet de conserver une structure claire tout en simulant le fonctionnement d'une base de données répartie entre plusieurs sites.

6.2 Database Links

Deux database links ont été créés depuis le schéma GLPI_GLOBAL afin de permettre l'accès aux données des schémas GLPI_CERGY et GLPI_PAU.

Ces liens Oracle permettent d'interroger des tables situées dans des schémas distants sans avoir besoin de dupliquer les données. Ils sont principalement utilisés par les vues globales pour centraliser les informations des deux campus au sein d'une même requête.

Dans le cadre du projet, les database links jouent un rôle clé dans l'architecture de la base de données répartie. Ils offrent au schéma GLPI_GLOBAL un accès transparent

aux données de chaque site tout en maintenant une séparation physique et logique entre les bases des campus de Cergy et de Pau.

6.3 Vues globales

Six vues globales ont été créées dans GLPI_GLOBAL pour consolider les données des deux sites :

Vue	Contenu
V_ALL_USERS	UNION ALL des utilisateurs Cergy et Pau
V_ALL_COMPUTERS	UNION ALL des matériels Cergy et Pau
V_ALL_TICKETS	UNION ALL des tickets Cergy et Pau
V_ALL_EQUIPEMENTS	UNION ALL des équipements réseau
V_ALL_HISTO	UNION ALL des historiques d'affectation
V_DASHBOARD_GLOBAL	Tableau de bord consolidé : nb users, matériels, tickets par site

6.4 Stratégie de fragmentation

La fragmentation est horizontale par site_id : chaque schéma ne contient que les données de son site. Les référentiels (rôles, types) sont répliqués dans les deux schémas car ils sont peu volumineux et rarement modifiés.

Type de donnée	Stratégie	Justification
Tickets, matériels, users	Fragmentation horizontale par site_id	Accès local rapide
Rôles, types, statuts	Réplication dans les deux schémas	Peu volumineux, stables
Vision globale	Vues UNION ALL via database links	Lecture consolidée

6.5 Vues matérialisées

En complément des vues classiques, trois vues matérialisées ont été créées dans le schéma GLPI_GLOBAL afin d'optimiser les requêtes analytiques les plus fréquentes :

- MV_DASHBOARD_GLOBAL : pré-calcule les principaux indicateurs par site (nombre total de tickets, tickets ouverts et tickets en cours) ;
- MV_STATS_TICKETS : pré-calcule les statistiques des tickets par site et par niveau de priorité ;
- MV_PARC_ACTIF : pré-calcule le nombre de matériels actifs par type et par site.

Contrairement aux vues classiques, qui interrogent les bases distantes à chaque exécution via les database links, les vues matérialisées stockent physiquement les résultats dans le schéma GLPI_GLOBAL. Cela permet de réduire le temps de réponse des requêtes analytiques en évitant de recalculer les statistiques à chaque exécution.

Les données des vues matérialisées sont mises à jour manuellement grâce à la procédure refresh_all_mviews, chargée de rafraîchir l'ensemble des vues matérialisées du schéma global.

7. Tests de Performance

7.1 Jeu de données

Les tests de performance ont été réalisés sur un volume représentatif généré par la procédure PL/SQL generate_all_data :

Table	Cergy	Pau
USERS	5000	5000
COMPUTERS	1000	1000
TICKETS	50 000	50 000
EQUIPEMENT_RESEAU	20	20
VLAN	10	10
HISTO_AFFECTATION	100	100

7.2 Résultats des requêtes

Les plans d'exécution ont été générés via EXPLAIN PLAN FOR + DBMS_XPLAN.DISPLAY sur glpi_cergy après génération du jeu de test complet.

Requête 1 - Matériels actifs par type :

```
... PLAN_TABLE_OUTPUT
```

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
0	SELECT STATEMENT		4	108	25 (8)	00:00:01
1	SORT ORDER BY		4	108	25 (8)	00:00:01
2	HASH GROUP BY		4	108	25 (8)	00:00:01
* 3	TABLE ACCESS FULL	TICKETS	2500	67500	23 (0)	00:00:01

Plan d'exécution EXPLAIN PLAN — Recherche des matériels actifs par type (TABLE ACCESS FULL, Cost=8)

Oracle choisit TABLE ACCESS FULL sur COMPUTERS et USERS avec HASH JOIN, Cost = 8. L'optimisation préfère le full scan car le volume (1000 computers) est jugé trop faible pour justifier un index. Ce comportement est attendu : en dessous d'un certain seuil, Oracle préfère le full scan. L'index composé devient déterminant avec des volumes supérieurs.

Requête 2 - Tickets ouverts par priorité :

```
.. PLAN_TABLE_OUTPUT
```

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
0	SELECT STATEMENT		4	108	25 (8)	00:00:01
1	SORT ORDER BY		4	108	25 (8)	00:00:01
2	HASH GROUP BY		4	108	25 (8)	00:00:01
* 3	TABLE ACCESS FULL	TICKETS	2500	67500	23 (0)	00:00:01

Plan d'exécution EXPLAIN PLAN — Tickets ouverts par priorité (TABLE ACCESS FULL, Cost=25)

Oracle effectue TABLE ACCESS FULL sur TICKETS (25 000 lignes, Cost = 25) avec HASH GROUP BY et SORT ORDER BY. Le prédicat filtre STATUS IN ('OPEN','IN_PROGRESS') AND SITE_ID = 1. Avec 50% des lignes retournées, l'optimiseur préfère le full scan. L'index composé serait bénéfique pour des volumes supérieurs ou des requêtes plus sélectives.

Requête 3 - Historique d'affectation :

```

*** PLAN_TABLE_OUTPUT
-----
| Id | Operation                | Name                | Rows | Bytes | Cost (%CPU)| Time      |
-----
|  0 | SELECT STATEMENT         |                     |    1 |    84 |    4 (0)| 00:00:01 |
|  1 |   NESTED LOOPS           |                     |    1 |    84 |    4 (0)| 00:00:01 |
|  2 |    NESTED LOOPS          |                     |    1 |    84 |    4 (0)| 00:00:01 |
|  3 |     NESTED LOOPS         |                     |    1 |    64 |    3 (0)| 00:00:01 |
|  4 |      TABLE ACCESS BY INDEX ROWID | COMPUTERS           |    1 |    19 |    1 (0)| 00:00:01 |
|*  5 |       INDEX UNIQUE SCAN    | SYS_C008370         |    1 |      |    0 (0)| 00:00:01 |
|  6 |      TABLE ACCESS BY INDEX ROWID | HISTO_AFFECTATION   |    1 |    26 |    2 (0)| 00:00:01 |
|*  7 |       INDEX RANGE SCAN DESCENDING | IDX_CERGY_HISTO_COMPUTER |    1 |      |    1 (0)| 00:00:01 |
|*  8 |       INDEX UNIQUE SCAN    | SYS_C008356         |    1 |      |    0 (0)| 00:00:01 |
|  9 |      TABLE ACCESS BY INDEX ROWID | USERS               |    1 |    20 |    1 (0)| 00:00:01 |
-----

```

Plan d'exécution EXPLAIN PLAN — Historique d'affectation (INDEX RANGE SCAN DESCENDING sur IDX_CERGY_HISTO_COMPUTER, Cost=4)

Cette requête démontre l'efficacité des index. Oracle utilise INDEX RANGE SCAN DESCENDING sur IDX_CERGY_HISTO_COMPUTER (Cost = 1), puis INDEX UNIQUE SCAN sur les clés primaires. Plan complet : NESTED LOOPS (Cost = 4) avec accès direct par index sur les 3 tables. Aucun full scan - accès optimal.

Requête 4 - Rapport curseur par priorité :

La procédure PL/SQL avec curseur OPEN/FETCH/CLOSE s'exécute en 0.1s sur 50 000 tickets : CRITICAL 1250 (12500 fermes), HIGH 1250 (12500 résolus), MEDIUM 1250 (12500 en cours), LOW 1250 (12500 ouverts). Distribution uniforme due à la génération cyclique.

7.3 Analyse comparative

Résultats sur glpi cergy :

Requête	Opération Oracle	Cost	Temps
Matériels actifs par type	TABLE ACCESS FULL + HASH JOIN	8	< 0.1s
Tickets ouverts par priorité	TABLE ACCESS FULL + HASH GROUP BY	25	< 0.1s
Historique affectation	INDEX RANGE SCAN DESCENDING	4	< 0.1s

Rapport curseur PL/SQL	HASH GROUP BY sur TICKETS	-	0.1s
BDDR COUNT Cergy+Pau	UNION ALL via database links	-	0.1s

Observation : l'optimiseur Oracle choisit le full scan pour les tables de taille modérée (300-500 lignes), comportement attendu et conforme à la théorie. L'index `IDX_CERGY_HISTO_COMPUTER` est utilisé efficacement (INDEX RANGE SCAN DESCENDING, Cost = 1) sur des recherches très sélectives. Les index composés deviendraient déterminants avec des volumes supérieurs à 10 000 lignes, typiques d'un vrai parc universitaire.

Afin de confirmer que les index composés sont bien opérationnels, nous avons forcé leur utilisation via un hint Oracle sur la requête de recherche des matériels actifs. Le plan d'exécution obtenu montre bien un INDEX RANGE SCAN sur `IDX_CERGY_COMPUTERS_SITE_TYPE_STATUS` avec un coût d'accès à l'index de 1, confirmant son bon fonctionnement.

```

***  PLAN_TABLE_OUTPUT
      Plan hash value: 4099516394

```

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
0	SELECT STATEMENT		173	20587	38 (0)	00:00:01
* 1	HASH JOIN		173	20587	38 (0)	00:00:01
2	TABLE ACCESS BY INDEX ROWID BATCHED	COMPUTERS	173	12456	19 (0)	00:00:01
* 3	INDEX RANGE SCAN	IDX_CERGY_COMPUTERS_SITE_TYPE_STATUS	173		1 (0)	00:00:01
4	TABLE ACCESS FULL	USERS	5000	229K	19 (0)	00:00:01

Plan d'exécution avec hint Oracle — Index composé `IDX_CERGY_COMPUTERS_SITE_TYPE_STATUS` forcé (INDEX RANGE SCAN, Cost index=1, Cost total=38)

Cependant, le coût total avec l'index forcé est de 38, contre 8 sans hint. Cela confirme que l'optimiseur Oracle avait raison de choisir le full scan pour ce volume de données : avec 1 000 ordinateurs, le parcours complet de la table est plus rapide que le parcours de l'index suivi des accès par rowid.

Ce comportement est cohérent avec la théorie Oracle : l'optimiseur préfère le full scan lorsque le ratio de lignes retournées est élevé par rapport au volume total. L'index composé deviendrait déterminant en production, avec un parc de plusieurs dizaines de milliers d'équipements répartis sur plusieurs années d'utilisation.

7.4 Comparaison Cergy vs Pau

Les tests sur glp_pau révèlent une différence notable sur la requête tickets par priorité. Sur Pau, Oracle utilise l'index composé IDX_PAU_TICKETS_SITE_STATUS avec INDEX RANGE SCAN (Cost = 5) au lieu du TABLE ACCESS FULL (Cost = 25 sur Cergy). Cela représente une réduction de coût de 80%.

Requête	Op. Cergy	Cost	Op. Pau	Cost	Analyse
Matériels actifs	TABLE ACCESS FULL	8	TABLE ACCESS FULL	8	Identique - volume faible
Tickets par priorité	TABLE ACCESS FULL	25	INDEX RANGE SCAN	5	Pau: gain 80% grâce à l'index
Historique affectation	INDEX RANGE SCAN DESC	4	INDEX RANGE SCAN DESC	3	Index utilise sur les 2 sites
Rapport curseur	HASH GROUP BY	0.1s	HASH GROUP BY	0.1s	Identique - procédure PL/SQL

Figure 5 : Plan d'exécution Pau — Tickets par priorité avec INDEX RANGE SCAN sur IDX_PAU_TICKETS_SITE_STATUS (Cost=5 vs Cost=25 sur Cergy)

La requête tickets sur Pau avec INDEX RANGE SCAN (Cost=5) vs TABLE ACCESS FULL sur Cergy (Cost=25) illustre l'apport concret des index composés : une réduction de 80% du coût estimé. Ce plan est optimal pour des requêtes sélectives sur des colonnes indexées.

7.5 Résultats BDDR - Vision globale

Les tests sur GLPI_GLOBAL confirment l'efficacité de l'architecture BDDR. Toutes les requêtes via database links s'exécutent en moins de 0.1s :

Requete BDDR	Cergy	Pau	Temps
COUNT users	5000	5000	0.0s
COUNT computers	1000	1000	0.0s
COUNT tickets	50 000	50 000	0.0s

V_DASHBOARD_GLOBAL	5000u / 1000C/ 50000t / 12500 ouverts	5000u / 1000C/ 50000t / 12500 ouverts	0.0s
V_ALL_TICKETS par priorité	4 x 12 500 tickets	4 x 12 500 tickets	0.0s

La vue V_DASHBOARD_GLOBAL retourne en temps réel les indicateurs clés des deux sites : 10 000 utilisateurs, 2 000 ordinateurs et 100 000 tickets au total dont 25 000 ouverts et 25 000 en cours. La fragmentation horizontale par site_id garantit que chaque schéma ne traite que ses propres données.

Test 7 : Dashboard Global

Vision consolidée Cergy + Pau

```
SELECT * FROM V_DASHBOARD_GLOBAL;
```

✓ 0.1s

SITE	NB_UTILISATEURS	NB_ORDINATEURS	NB_TICKETS_TOTAL	TICKETS_OUVERTS	TICKETS_EN_COURS	MATERIEL_ACTIF
CERGY	5000	1000	50000	12500	12500	867
PAU	5000	1000	50000	12500	12500	867

V_DASHBOARD_GLOBAL — Vision consolidée Cergy + Pau en temps réel (0.1s)

7.6 Vues Matérialisées vs Vues classiques

L' EXPLAIN PLAN compare l'exécution de V_DASHBOARD_GLOBAL (vue normale) et MV_DASHBOARD_GLOBAL (vue matérialisée) sur GLPI_GLOBAL après génération de 100 000 tickets :

Objet	Opérations Oracle	Cost	Appels réseau
V_DASHBOARD_GLOBAL	UNION-ALL + 12 REMOTE (6 Cergy + 6 Pau)	4	12 appels distants
MV_DASHBOARD_GLOBAL	MAT_VIEW ACCESS FULL (lecture locale)	3	0 appel réseau

```
EXPLAIN PLAN FOR
SELECT * FROM V_DASHBOARD_GLOBAL;
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
```

✓ 0.1s

Explained.

PLAN_TABLE_OUTPUT

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time	Inst	IN-OUT
0	SELECT STATEMENT		2	164	4 (0)	00:00:01		
1	VIEW	V_DASHBOARD_GLOBAL	2	164	4 (0)	00:00:01		
2	UNION-ALL							
3	REMOTE						CERGY~	R->S
4	REMOTE						CERGY~	R->S
5	REMOTE						CERGY~	R->S
6	REMOTE						CERGY~	R->S
7	REMOTE						CERGY~	R->S
8	REMOTE						CERGY~	R->S
9	FAST DUAL		1		2 (0)	00:00:01		
10	REMOTE						PAU_L~	R->S
11	REMOTE						PAU_L~	R->S
12	REMOTE							PAU_L~ R->S
13	REMOTE							PAU_L~ R->S
14	REMOTE							PAU_L~ R->S
15	REMOTE							PAU_L~ R->S
16	FAST DUAL			1		2 (0)	00:00:01	

EXPLAIN PLAN — V_DASHBOARD_GLOBAL (12 appels REMOTE, Cost=4)

```
EXPLAIN PLAN FOR
SELECT * FROM MV_DASHBOARD_GLOBAL;
SELECT * FROM TABLE(DBMS_XPLAN.DISPLAY);
```

✓ 0.1s

Explained.

PLAN_TABLE_OUTPUT

Plan hash value: 2312925604

Id	Operation	Name	Rows	Bytes	Cost (%CPU)	Time
0	SELECT STATEMENT		2	32	4 (0)	00:00:01
1	MAT_VIEW ACCESS FULL	MV_DASHBOARD_GLOBAL	2	32	4 (0)	00:00:01

MV_DASHBOARD_GLOBAL (MAT_VIEW ACCESS FULL local, Cost=3)

La vue matérialisée élimine les 12 appels réseau distants (REMOTE) en pré-calculant les données localement. Le plan V_DASHBOARD_GLOBAL montre 6 accès REMOTE vers cergy_link et 6 vers pau_link à chaque exécution. MV_DASHBOARD_GLOBAL effectue un simple MAT_VIEW ACCESS FULL local (Cost=3 vs Cost=4). En production avec une latence réseau Q réelle entre Cergy et Pau, ce gain serait très significatif.

7.7 Limites du projet

Les tests réalisés dans le cadre du projet ont été effectués sur une instance locale Oracle XE, ce qui limite l'impact réel de la latence réseau entre les sites de Cergy et de Pau.

De plus, bien que le jeu de données généré soit conséquent dans un contexte pédagogique, les volumes restent inférieurs à ceux d'un environnement de production réel.

Enfin, certains aspects avancés de sécurité, de haute disponibilité et de réplication Oracle n'ont pas été étudiés dans le périmètre du projet.

8. Conclusion

Ce projet nous a permis de concevoir et d'implémenter une base de données Oracle complète inspirée du reverse engineering de GLPI, adaptée au contexte multi-sites de CY Tech.

Les principaux apports par rapport à la base GLPI originale sont :

- Une architecture BDDR claire avec fragmentation horizontale par site_id et consolidation via database links
- Des tablespaces dédiés séparant données, index et historiques pour une meilleure gestion du stockage
- Des index composés alignés sur les patterns de requêtes CY Tech, réduisant les full table scans
- Un cluster Oracle regroupant physiquement les tickets avec leurs matériels
- Des triggers garantissant l'intégrité métier (traçabilité des affectations, protection des tickets ouverts)
- Des fonctions et procédures PL/SQL avec curseurs automatisant les opérations courantes
- Un jeu de 100 000 tickets + 10 000 utilisateurs + 2 000 matériels pour valider les performances

Les tests de performance montrent l'efficacité des index composés sur les requêtes multi-critères fréquentes dans un contexte de gestion de parc informatique multi-sites. Les résultats obtenus mettent également en évidence le comportement de l'optimiseur Oracle, qui adapte automatiquement les plans d'exécution en fonction du volume de données et de la sélectivité des requêtes. Les optimisations mises en place deviennent ainsi particulièrement pertinentes lorsque le volume de données augmente et que les requêtes deviennent plus ciblées.